

数学实验与数学软件

第六课：MATLAB数值微积分

谭军

中山大学数学学院

教师邮箱：mcstj@mail.sysu.edu.cn



复习-符号运算-计算导数

□ diff

$g = \text{diff}(f, v)$: 求符号表达式 f 关于 v 的导数

$g = \text{diff}(f)$: 求符号表达式 f 关于默认变量的导数

$g = \text{diff}(f, v, n)$: 求 f 关于 v 的 n 阶导数

- 这里的 f 为符号对象或符号变量，所求得的导数也将由符号表达式进行存储和表示
- 对矩阵表达式求（偏）导数，即对矩阵的每一个元素表达式分别求（偏）导数
- 可使用 $f_{\text{jac}} = \text{jacobian}(f, v)$ 对矩阵表达式求其雅可比矩阵，此时的 f 与 v 需要定义成函数向量与自变量向量

差分方法-计算近似导数

- $\mathbf{dx}=\text{diff}(\mathbf{X})$ 为向量或矩阵的向前差分运算，即最直接的用向量的后面的元素减去前面相邻的元素求差，或矩阵的最后一行与前面相邻的行求差。该函数等价于 $\mathbf{dx}=\mathbf{X}(2:n, :)-\mathbf{X}(1:n-1, :)$ ，注意dx的行数要比X少一行。
- $\mathbf{FX}=\text{gradient}(\mathbf{F})$ 称为向量或矩阵的中心差分运算，即选取当前元素的下一个元素，减去上一个元素，再除以2，这种方法的近似精度比向前差分要高。具体来说本函数等价于 $\mathbf{FX}(1)=\mathbf{F}(2)-\mathbf{F}(1)$ ， $\mathbf{FX}(\text{end})=\mathbf{F}(\text{end})-\mathbf{F}(\text{end}-1)$ ， $\mathbf{FX}(2:\text{end}-1)=(\mathbf{F}(3:\text{end})-\mathbf{F}(1:\text{end}-2))/2$ 。可以观察到FX与F长度相同。矩阵的“一元梯度”仅包括横向
- $[\mathbf{FX}, \mathbf{FY}]=\text{gradient}(\mathbf{F})$ 称为二元梯度运算，FX为横向梯度（每一行右减左），FY为纵向梯度（每一列下减上）。

例4.1-1

【例 4.1-1】设 $f_1(x) = \frac{1 - \cos 2x}{x \sin x}$, $f_2(x) = \frac{\sin x}{x}$, 试用最小正数 `realmin` 替代理论 0 计算极限 $L_1(0) = \lim_{x \rightarrow 0} f_1(x)$,

$$L_2(0) = \lim_{x \rightarrow 0} f_2(x)。$$

```
x=realmin;
```

```
L1=(1-cos(2*x))/(x*sin(x)), %分子分母太小, 出现0/0
```

```
L1 = NaN
```

```
L2=sin(x)/x,
```

```
%L2 = 1
```

```
syms t
```

```
f1=(1-cos(2*t))/(t*sin(t));
```

```
f2=sin(t)/t;
```

```
Ls1=limit(f1,t,0)
```

%符号运算的结果是精确的

```
Ls1 = 2
```

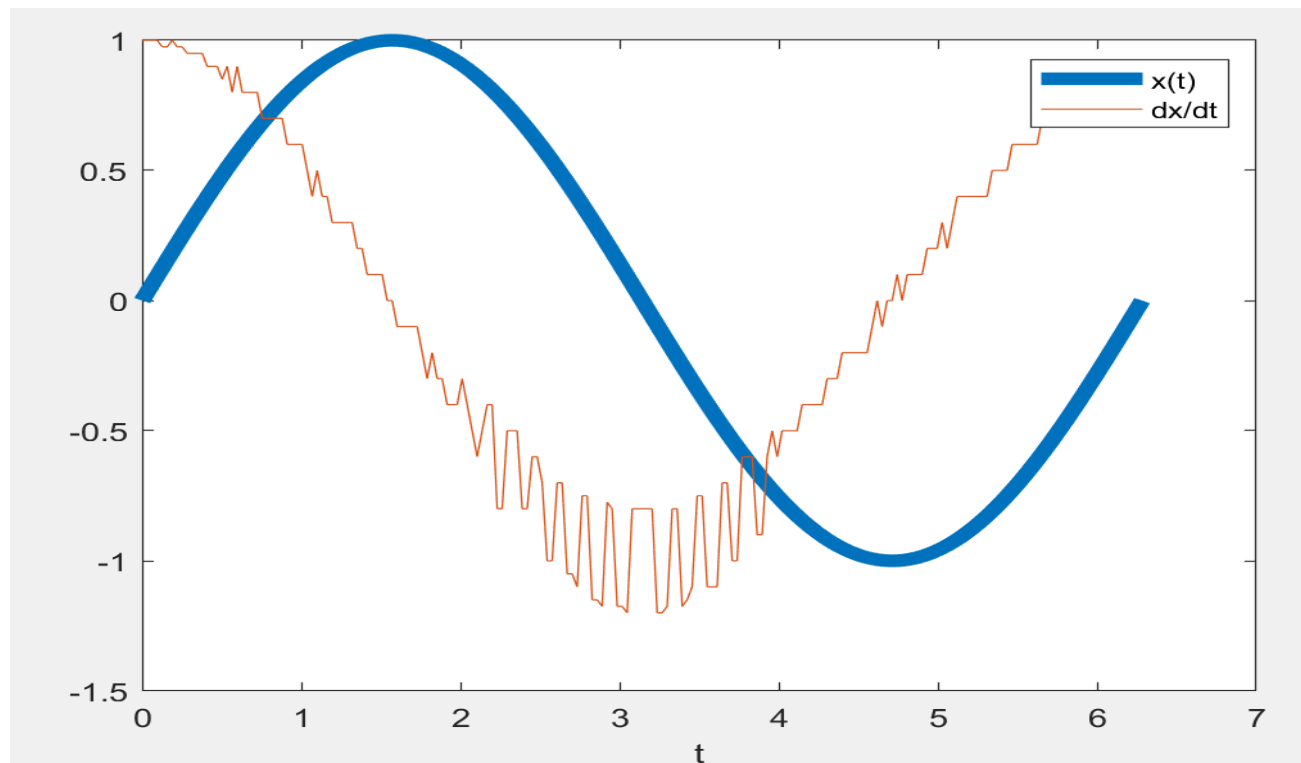
```
Ls2=limit(f2,t,0)
```

```
Ls2 = 1
```

例4.1-2

【例 4.1-2】求 $x = \sin(t)$ 在 $[0, 2\pi]$ 中的近似导函数

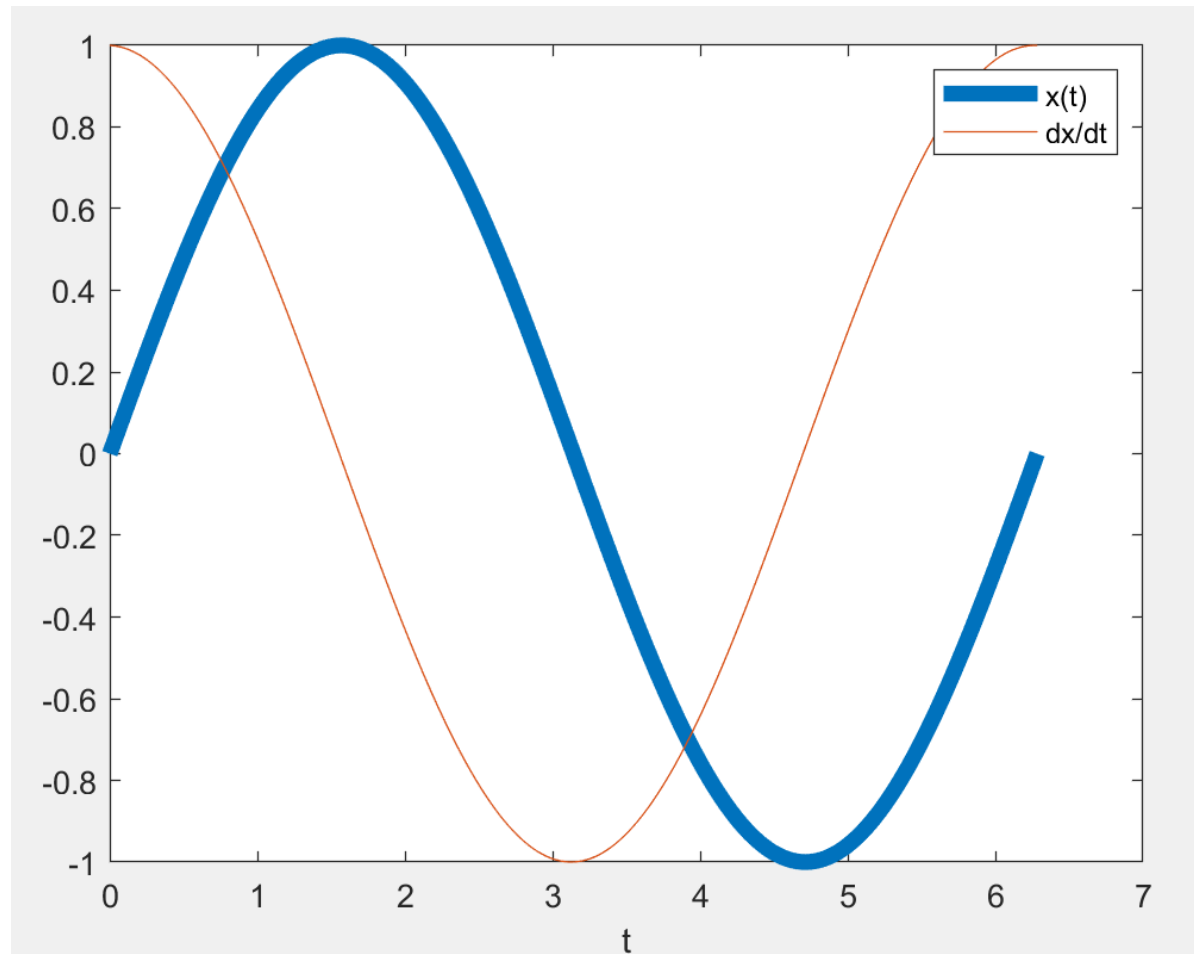
```
d=pi/100; %采样点步长为pi/100,步长越细,曲线越清晰
t=0:d:2*pi;x=sin(t);
dt=5*eps; %求导跨度一般是越小越好,但太小了会出现“除小毛刺”
x_eps=sin(t+dt);
dxdt_eps=(x_eps-x)/dt;%相较diff函数,跨度dt远远小于d,除法会丢失有效数字
plot(t,x,'LineWidth',5)%画函数
hold on,plot(t,dxdt_eps),hold off%画导数
legend('x(t)', 'dx/dt'),xlabel('t')
```



例4.1-2

【例 4.1-2】求 $x = \sin(t)$ 在 $[0, 2\pi]$ 中的近似导函数

```
x_d=sin(t+d);  
dxdt_d=(x_d-x)/d; %其实以d作为增量基本上已经够了  
plot(t,x,'LineWidth',5)  
hold on,plot(t,dxdt_d),hold off  
legend('x(t)', 'dx/dt'),xlabel('t')
```



例4.1-3

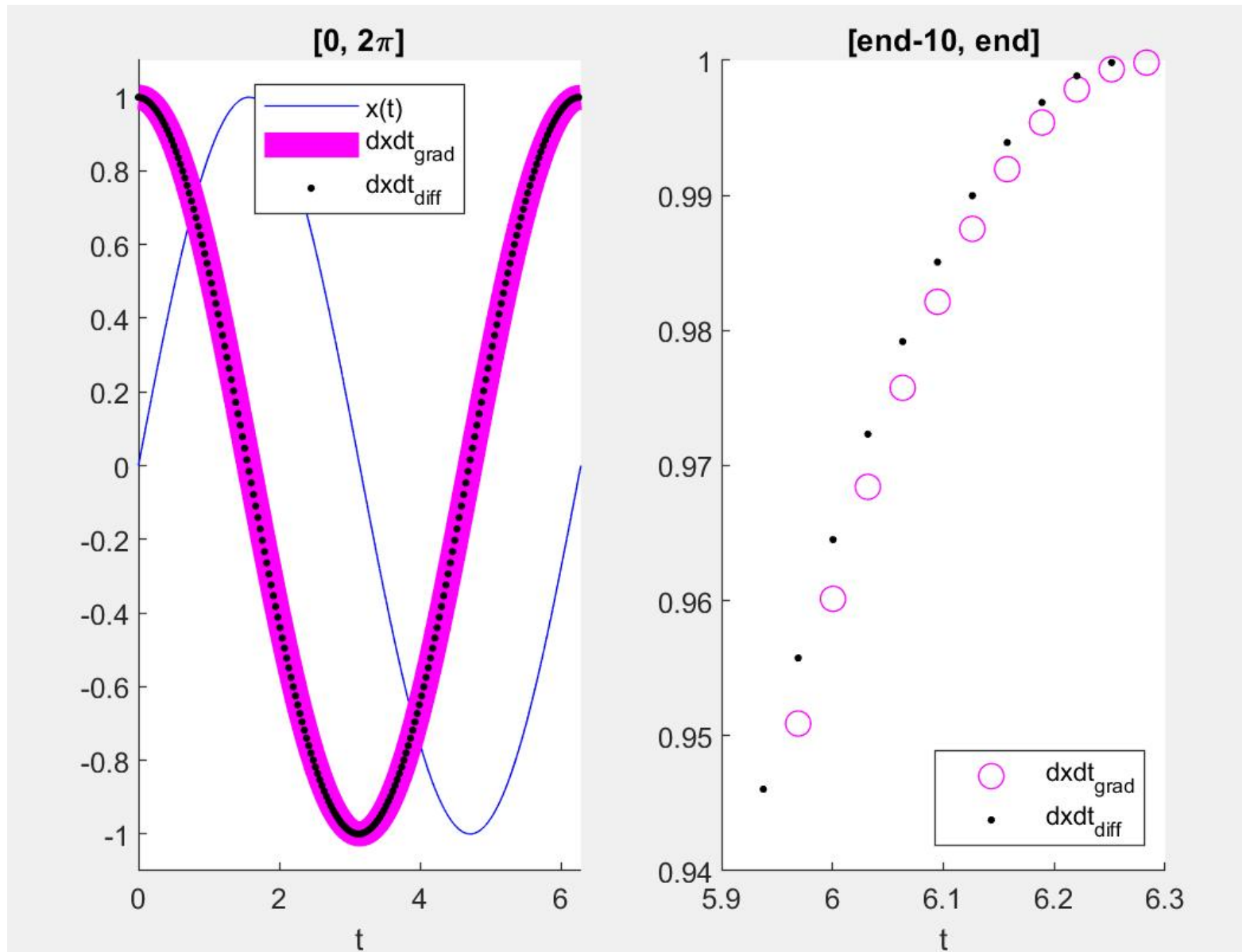
【例 4.1-3】利用函数 diff 与 gradient 计算 $x = \sin(t)$ 在 $t \in [0, 2\pi]$ 的导数

```
clf
d=pi/100; %步长仍为pi/100
t=0:d:2*pi;x=sin(t);
dxdt_diff=diff(x)/d; %向前差分
dxdt_grad=gradient(x)/d; %中心差分
subplot(1,2,1),plot(t,x,'b')
hold on,plot(t,dxdt_grad,'m','LineWidth',8) %品红色 (Magenta) 粗线
plot(t(1:end-1),dxdt_diff,'.k','MarkerSize',8)
%黑色点,注意两条线的绘画顺序不可调换,否则更粗的品红色线将覆盖这些黑点
axis([0,2*pi,-1.1,1.1]),title('[0, 2\pi]'),
legend('x(t)','dxdt_{grad}','dxdt_{diff}','Location','North')
xlabel('t'),box off,hold off

subplot(1,2,2)
kk=(length(t)-10):length(t); %考察最后十一个点的值 (diff少一个)
hold on,plot(t(kk),dxdt_grad(kk),'om','MarkerSize',8)
plot(t(kk-1),dxdt_diff(kk-1),'.k','MarkerSize',8)
title('[end-10, end]')
legend('dxdt_{grad}','dxdt_{diff}','Location','SouthEast')
xlabel('t'),box off,hold off
```

例4.1-3

【例 4.1-3】利用函数 diff 与 gradient 计算 $x = \sin(t)$ 在 $t \in [0, 2\pi]$ 的导数



数值求和与累积和

- `sum(X)` 或 `sum(X,1)` 为矩阵按列方向求和,即将所有列的和存储到一个行向量中。但若X为行向量, `sum`也将按行求和。
- `sum(X,2)` 为矩阵按行方向求和,即将所有行的和存储到一个列向量中。
- `Scs=cumsum(X)` 为矩阵按列方向求累积和,结果矩阵Scs与X同型,值为X从上到下的累积 $Scs(m,n) = \sum_{i=1}^m X(i,n)$

□ `>>A=[1,2;3,4]`

`>>S1=sum(A) , S2=sum(A,2) , S3=cumsum(A)`

A=	S1=	S2=	S3=
1 2	4 6	3	1 2
3 4		7	4 6

数值积分-(复合)矩形公式

- 由微积分基本定理，若 $F(x)$ 为 $f(x)$ 的一个原函数，则：

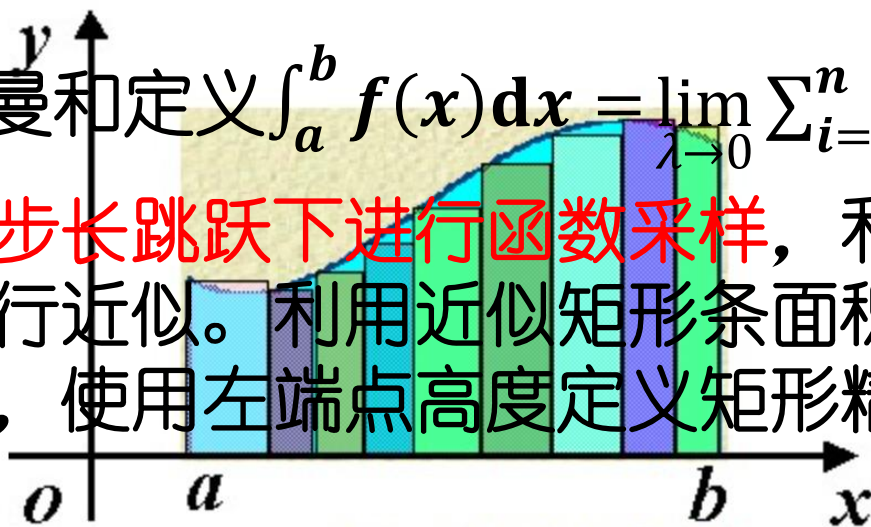
$$\int_a^b f(x) dx = F(b) - F(a)$$

- 很多时候，一个函数的原函数无法用初等函数表示，或表示极为复杂，如： $\int \frac{1}{1+x^6} dx =$

$$\frac{1}{3} \tan^{-1} x + \frac{1}{6} \tan^{-1} \left(x - \frac{1}{x} \right) + \frac{1}{4\sqrt{3}} \ln \frac{x^2 + x\sqrt{3} + 1}{x^2 - x\sqrt{3} + 1} + C$$

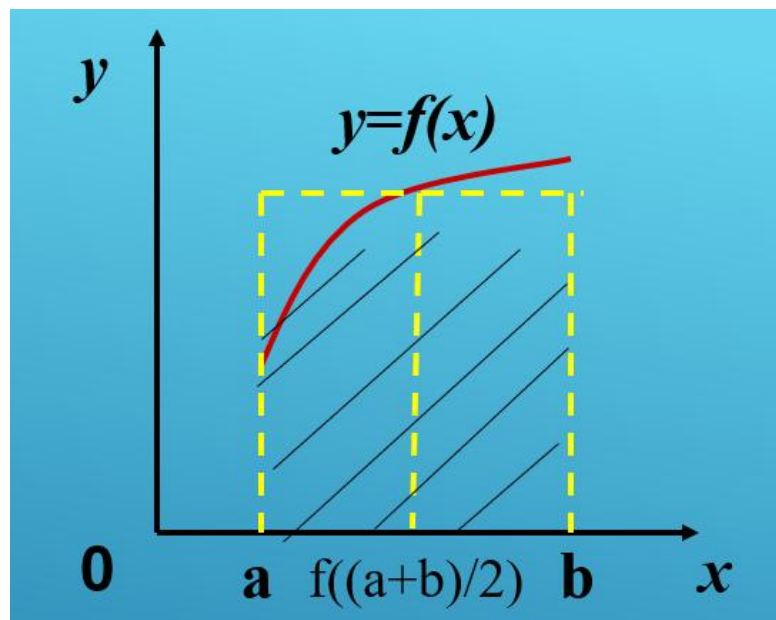
- 基于定积分黎曼和定义 $\int_a^b f(x) dx = \lim_{\lambda \rightarrow 0} \sum_{i=1}^n f(\xi_i) \Delta x_i$,

对函数在较小的步长跳跃下进行函数采样，利用所求的和，对定积分的值进行近似。利用近似矩形条面积求和的公式被称之为矩形公式，使用左端点高度定义矩形精度相对较差。

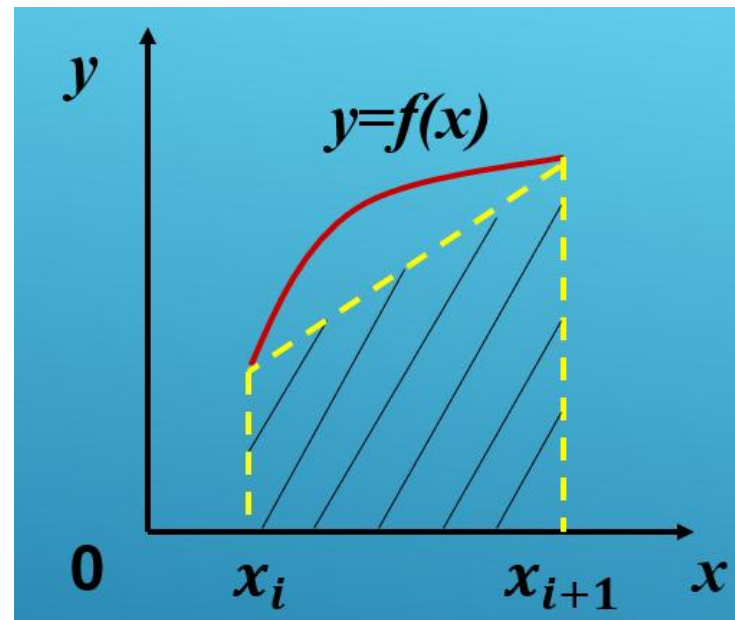


数值积分-(复合)梯形公式

- 将定积分 $\int_a^b f(x)dx$ 首先分为若干个积分节点, $a = x_0 < x_1 < x_2 < \dots < x_n = b$, 对每一个节点分段 $[x_i, x_{i+1}]$, 分段积分 $\int_{x_i}^{x_{i+1}} f(x)dx$ 除了用矩形公式 $f(\xi_i)(x_{i+1} - x_i)$ (ξ_i 可取左端点 x_i , 右端点 x_{i+1} , 中点 $\frac{x_i + x_{i+1}}{2}$) 来近似之外, 还可以用梯形公式 $\frac{f(x_i) + f(x_{i+1})}{2} (x_{i+1} - x_i)$ 来近似。



中矩形公式



梯形公式

梯形公式的算法实现

【例】利用梯形公式近似计算定积分 $\int_0^1 \frac{1}{1+x^2} dx$,并利用计算结果估计常数 π 的值

□ 尝试对闭区间 $[0, 1]$ 以 1×10^{-5} 为步长进行均匀采样

```
clear,d = 1e-5; %设定步长d为1x10^(-5)
x = 0:d:1;
fx = 1./(1+x.^2); %计算所有采样点的函数值
fm = (fx(1:end-1)+fx(2:end))/2;
%计算（采样点数-1）个区间的左右端点函数均值

f_int = sum(fm)*d; %理论结果pi/4
pi_app = f_int*4
err = abs(pi_app - pi) pi_app = 3.1416
err = 1.6669e-11
```

□ 对于某些实际的数值积分问题，被积函数可能本身就没有规范的函数型定义，最初的数据就是离散的采样点序列 $\{x_i\}_{0 \leq i \leq n}$ 及其对应的函数值列表 $\{f(x_i)\}_{0 \leq i \leq n}$ ，利用梯形公式同样可以计算其数值积分，相当于进行分片线性插值后，再计算了定积分的值。

梯形公式的算法实现

【例】利用梯形公式近似计算定积分 $\int_0^1 \frac{1}{1+x^2} dx$,并利用计算结果估计常数 π 的值

□ 更改步长到 1×10^{-8} , 观察误差的变化

```
clear,d = 1e-8; %设定步长d为1x10^(-8)
x = 0:d:1;
fx = 1./(1+x.^2); %计算所有采样点的函数值
fm = (fx(1:end-1)+fx(2:end))/2;
%计算（采样点数-1）个区间的左右端点函数均值

f_int = sum(fm)*d; %理论结果pi/4
pi_app = f_int*4
err = abs(pi_app - pi)
format long,pi_app,format
pi_app = 3.141592653589910
pi_app = 3.1416
err = 1.1680e-13
%前12位小数是基本可以采信
```

□ 步长的减少增加了运算区间的数量, 提高了运算精度, 但也对运算的时间和空间成本提出了更高的要求

梯形公式的MATLAB函数

- ❑ **trapz(y)** 将使用梯形公式方法，对函数矩阵y按列方向求和，即每一列的第一个和最后一个元素在求和时需乘以权值0.5（为什么？）。若y为行向量，trapz也会按行计算
- ❑ **trapz(x,y)** 为函数矩阵y按列方向求关于x的和。此时，x为函数采样序列y对应的自变量列表。计算求和时，每一块梯形区域将自动乘以对应的底边宽度。允许x不均匀采样。
- ❑ **cumtrapz(y)** 为函数矩阵累积式按列方向求和，结果与y为同型矩阵，计算原理可类比MATLAB函数**cumsum**。
- ❑ **cumtrapz(x,y)** 同样为函数矩阵累积式按列方向求和，x指定了自变量列表，计算原理可类比MATLAB函数**cumsum**。

例4.1-4

【例 4.1-4】计算 $\int_0^{\pi/2} 0.2 + \sin t \, dt$

```
clear,d=pi/8;    t=0:d:pi/2;    y=0.2+sin(t);
```

```
s=sum(y);
```

%按照矩形公式求和，实际上多加了一个点

```
s_sa=d*s;
```

%矩形公式和乘以步长可以近似定积分值

```
s_ta=d*trapz(y);
```

%可直接替换为s_ta=trapz(t,y)

```
disp(['sum求得积分',blanks(3),'trapz求得积分'])
```

```
disp([s_sa, s_ta])
```

```
sum求得积分    trapz求得积分
```

```
1.5762
```

```
1.3013
```

```
t2=[t,t(end)+d];y2=[y,nan];
```

%多花一层阶梯表明sum多算

```
hs=stairs(t2,y2,':k','LineWidth',3);
```

%绘制阶梯折线，与plot类似

```
hold on
```

```
ht=plot(t,y,'r','LineWidth',3);
```

%函数曲线绘制

```
stem(t,y)
```

%分布杆图，空心小圆圈标注点，有竖条

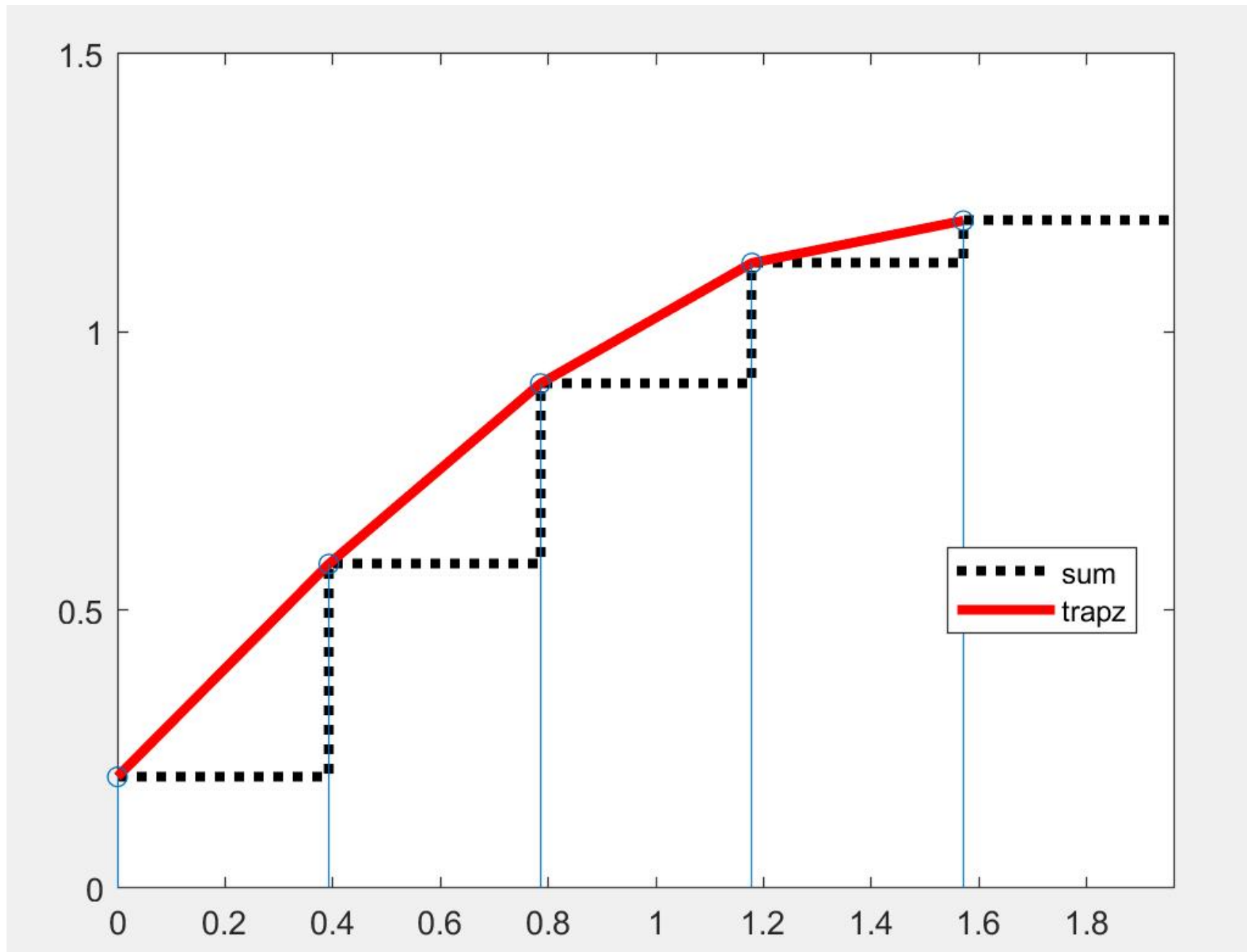
```
legend([hs,ht],'sum','trapz','location','best')%best位置（智能）
```

```
axis([0,pi/2+d,0,1.5])
```

```
hold off
```

```
shg %绘图窗口置顶，立即显示
```

例4.1-4



数值积分-(复合)辛普森公式

□ 梯形公式: $\int_{x_i}^{x_{i+1}} f(x) dx \approx \frac{f(x_i) + f(x_{i+1})}{2} (x_{i+1} - x_i)$

□ 辛普森公式:

$$\int_{x_i}^{x_{i+1}} f(x) dx \approx \frac{f(x_i) + 4f\left(\frac{x_i + x_{i+1}}{2}\right) + f(x_{i+1})}{6} (x_{i+1} - x_i)$$

□ 梯形公式的代数精度（误差之于步长的阶数）明显低于辛普森公式，因此使用辛普森公式可以在步长相同的情况下，显著的降低数值积分的误差。

辛普森公式的算法实现

【例】利用辛普森公式近似计算定积分 $\int_0^1 \frac{1}{1+x^2} dx$, 并利用计算结果估计常数 π 的值

□ 尝试对闭区间 $[0, 1]$ 以 1×10^{-5} 为步长进行均匀采样

```
clear,d = 1e-5; %设定步长d为1x10^(-5)
x = 0:d:1;
xh = x+d/2; %定义半步长采样点
fx = 1./(1+x.^2); %计算所有全步长采样点的函数值
fxh = 1./(1+xh.^2); %计算所有半步长采样点的取值
fm = (fx(1:end-1)+4*fxh(1:end-1)+fx(2:end))/6;
%代入辛普森公式, 计算(全步长采样点数-1)个区间近似积分值

f_int = sum(fm)*d; %理论结果pi/4
pi_app = f_int*4
err = abs(pi_app - pi)
format long,pi_app,format
pi_app = 3.141592653589794
pi_app = 3.1416
err = 1.33239e-15
%前14位小数是基本可以采信的
```

□ 可见辛普森公式的精度大大超过梯形公式, 步长可以更长

数值积分-蒙特卡洛法

□ 积分计算公式:

$$\int_a^b f(x)dx \approx \frac{\sum_{j=1}^K f(x_j)}{K} (b - a),$$

其中, $\forall 1 \leq j \leq K, x_j \in [a, b]$, 为区间随机采样点, 采样点的选取一般采用均匀分布, 根据实际需要, K 的值往往会不少于 1×10^4 。有时, 实际问题会提供一些不知道函数采样位置的随机函数采样, 直接计算均值再乘以区间长度, 等效于蒙特卡洛法。

□ 蒙特卡洛法同样可以推广到复合型, 子区间积分计算公式:

$$\int_{x_i}^{x_{i+1}} f(x)dx \approx \frac{\sum_{j=1}^K f(x_{i,j})}{K} (x_{i+1} - x_i),$$

其中, $\forall 1 \leq j \leq K, x_{i,j} \in [x_i, x_{i+1}]$, 为子区间随机采样点, 采样点的选取一般采用均匀分布, 区间内选点个数 K 的值往往与需求的误差和步长有关, 步长越长, 选点个数越多。(不推荐此方法因随机采样与固定区间划分有一定矛盾, 算法复杂度会很高)

蒙特卡洛法的算法实现

【例】利用蒙特卡洛法(1×10^5 个采样点)近似计算定积分 $\int_0^1 \frac{1}{1+x^2} dx$,并利用计算结果估计常数 π 的值

```
clear,Nsp=1e5;  
x=rand(1,Nsp);  
fx = 1./(1+x.^2);
```

%总采样点1x10^5个

%计算所有采样点的函数值

```
f_int = sum(fx)/Nsp;  
%求和取平均, 区间长度1, 理论结果pi/4
```

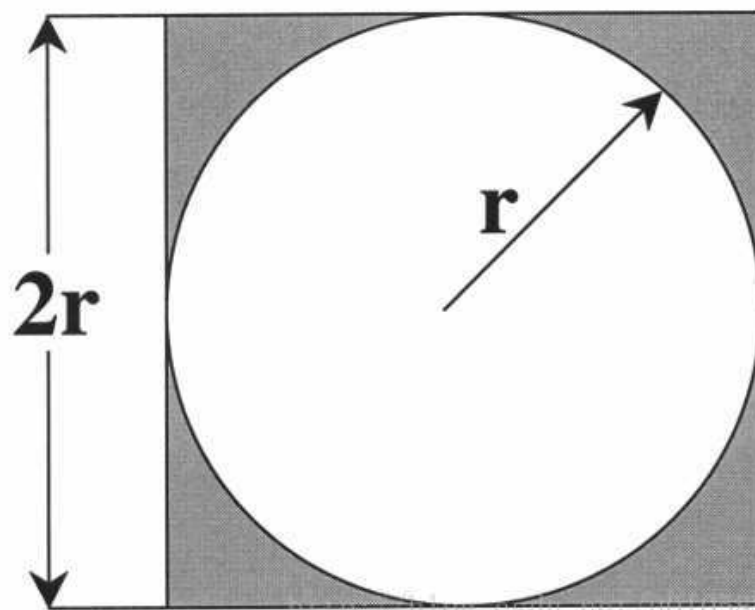
```
pi_app = f_int*4  
err = abs(pi_app - pi)
```

```
pi_app = 3.1421  
err = 4.8739e-04
```

- ❑ 连续测试蒙特卡洛法, 误差会有所不同, 这是由于采样点随机选取的原因。
- ❑ 蒙特卡洛法误差偏大, 尤其不适合计算局部绝对值很大的被积函数, 但实际应用中, 未知采样位置的采样数据非常多见, 因此该方法仍然在特定问题上有重要意义。

蒙特卡洛法的其他应用

- 一维蒙特卡洛法估计实数的值，例： $\sqrt{2}$ 的一种估计方法
首先，假设随机样本 $X \sim U[0, 2]$ ，这种均匀分布的样本可以通过生成`rand`矩阵后，再将矩阵元素整体乘以2
- 而后在大量的随机数据（共 N 个）中，统计所有满足 $x^2 \leq 2$ 的数据并计数为 N_a 个，此时，我们可以近似的估计 $\sqrt{2} = 2 \cdot \frac{N_a}{N}$
- 二维蒙特卡洛法估计实数的值，例： π 的估计，如右下图，若设 $r = \frac{1}{2}$ ，且圆心为 $(\frac{1}{2}, \frac{1}{2})$ 可生成大量（ N 组）符合 $(0, 1)$ 均匀分布的横坐标 X 与纵坐标 Y 。易知 (X, Y) 恰好落在正方形区域内。此时统计落在圆形区域 $(x - \frac{1}{2})^2 + (y - \frac{1}{2})^2 \leq \frac{1}{4}$ 内的点数 N_a ，即可近似估计 $\pi \approx 4 \cdot \frac{N_a}{N}$



匿名函数

- 适用于结构简单、无需创建m文件或子函数体（后面会讲）来定义的“一句话函数”
- 例如： $F=@(x) \sin(x) .* x$ ，就定义了一个自变量为 x ，函数值 $F(x)=\sin(x) .* x$ 的匿名函数。命名上， F 称为函数句柄，括号内的 x 称为参数（参数可以由多个变量构成参数列表）， $\sin(x) .* x$ 称为函数主体表达式。
- 匿名函数可以通过如 $F(3)$ 或 $F(x)$ 直接调用，也可以通过 $\text{feval}(F,x)$ 间接调用，有时，函数句柄 F 还可以作为参数代入一些更为复杂的函数体。

MATLAB内建的数值积分函数

- ❑ `Q=integral(fun,xmin,xmax)`，其中`fun`为匿名函数句柄或其它函数句柄，`xmin`与`xmax`分别代表积分的下限与上限。默认的绝对误差上限为 1×10^{-10} ，默认相对误差上限为 1×10^{-6} 。
- ❑ 使用的方法可能包括自适应辛普森法，自适应罗巴托法，等各种高端算法，（代码不可见 😊）
- ❑ `Q=integral(fun,xmin,xmax,'AbsTol',1e-12)` 可以将绝对误差的上限进一步降低两个数量级
- ❑ `Q=integral(fun,xmin,xmax,'ArrayValued',true)`
代表被积函数为阵列函数，或者被积函数包含多个函数。或者被积函数“只能逐点代入运算”，不能以列向量整体运算，如`F=@(x) sin(x)*x`就必须使用阵列函数运算。否则需改为`F=@(x) sin(x).*x`

MATLAB内建的数值积分函数

- ❑ `Q=integral(fun,xmin,xmax,'Waypoints',[1+1i,1-1i])` 代表积分路径被指定，多用于复平面上的指定路径的积分运算。
- ❑ `integral` 函数支持广义积分，使用方法与符号函数类似。发散的广义积分可能会出现计算的错误或警告。无穷积分的范围与符号运算类似，`Inf`或`inf`代表正无穷，`-Inf`或`-inf`代表负无穷。而瑕积分无需对瑕点进行特别处理。
- ❑ 本课程既要求直接学会使用MATLAB内建函数，也需要掌握使用复合矩形、复合梯形公式或蒙特卡洛法独立完成数值积分的运算，同时理解其他的方法的含义。

例4.1-5 (求代数和面积)

【例 4.1-5】在 $0 \leq x \leq 1$ 之间，求曲线 $g_1(x) = \frac{1}{\sqrt[5]{x}}$ 与 $g_2(x) = x^5$ 所夹区域的面积

```
x=linspace(0.01,1.2,50);
```

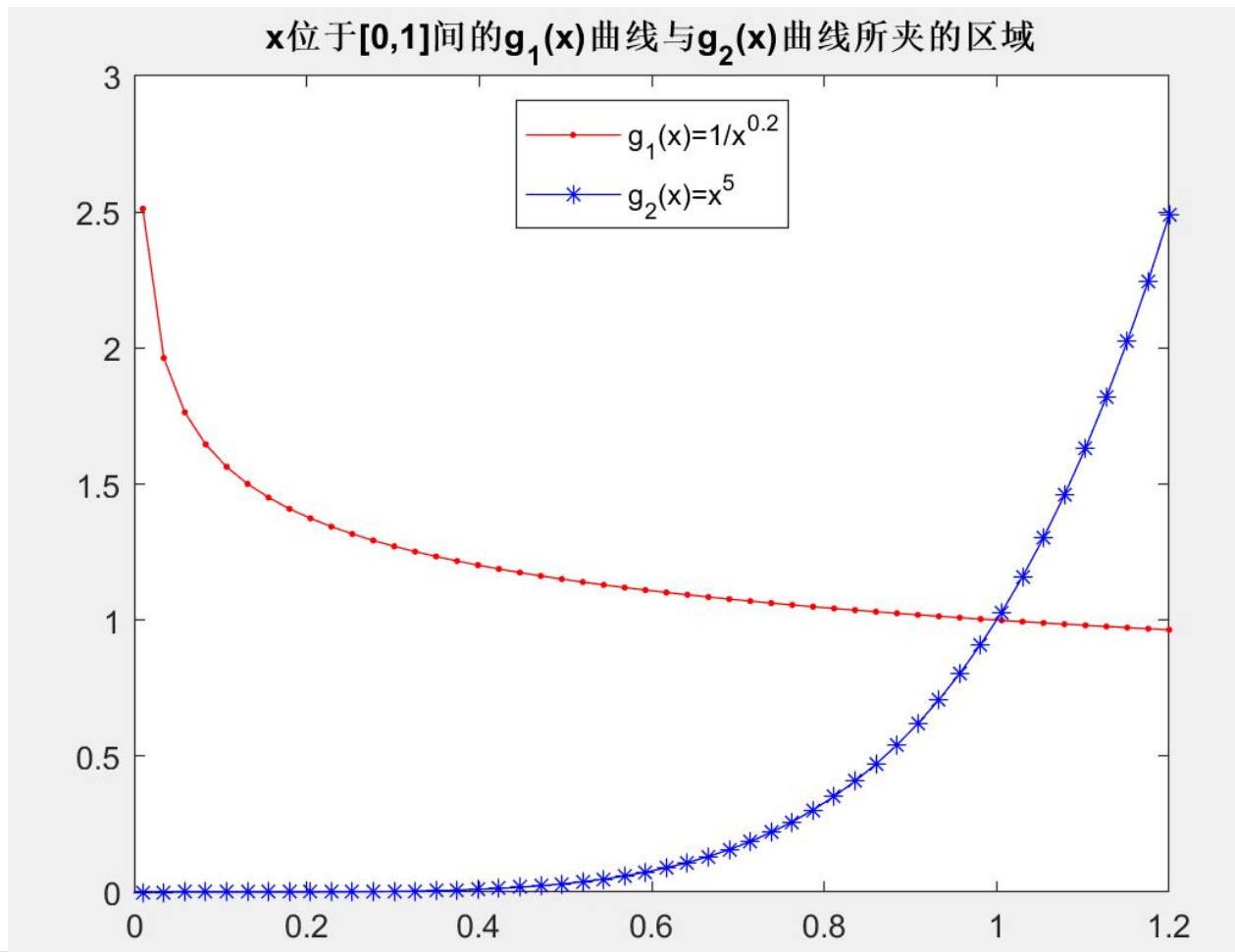
%瑕积分因此选点从0.01开始

```
g1=x.^(-0.2); g2=x.^5;
```

```
plot(x,g1,'-r.',x,g2,'-b*'), axis([0,1.2,0,3])%两个函数的绘图
```

```
legend('g_1(x)=1/x^{0.2}','g_2(x)=x^5','Location','North')
```

```
title('x位于[0,1]间的g_1(x)曲线与g_2(x)曲线所夹的区域')
```



例4.1-5

【例 4.1-5】在 $0 \leq x \leq 1$ 之间，求曲线 $g_1(x) = \frac{1}{\sqrt[5]{x}}$ 与 $g_2(x) = x^5$ 所夹区域的面积

```
format long
```

```
G1=@(x)x.^-0.2;
```

```
Q1=integral(G1,0,1)
```

%匿名函数g1，注意使用.^

```
Q1 = 1.250000027856048
```

```
G2=@(x)x.^5;
```

```
Q2=integral(G2,0,1)
```

%匿名函数g2

```
Q2 = 0.1666666666666667
```

```
S12=Q1-Q2
```

```
S12 = 1.083333361189381
```

%计算积分的差，即面积的代数和(有符号)

```
G=@(x)x.^[-0.2;5];
```

```
Q=integral(G,0,1,'ArrayValued',true)
```

%定义了阵列型匿名函数

```
Q =
```

```
1.250000027856048
```

```
0.1666666666666667
```

```
S=[1,-1]*Q
```

```
S =
```

```
1.083333361189381
```

例4.1-5, 6

【例 4.1-5】在 $0 \leq x \leq 1$ 之间，求曲线 $g_1(x) = \frac{1}{\sqrt[5]{x}}$ 与 $g_2(x) = x^5$ 所夹区域的面积

syms t

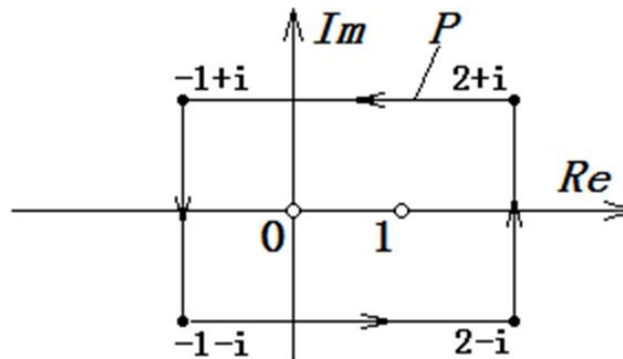
```
Gsym=vpa(int(t.^[-0.2;5],0,1));
```

$$S_{\text{sym}} = G_{\text{sym}}(1) - G_{\text{sym}}(2)$$

31位

$$S_{\text{sym}} = 1.08333333333333333333333333333333$$

【例 4.1-6】验证 $s = \oint_P \frac{2z-1}{z(z-1)} dz = 4\pi i$ 。



```
F=@ (z) (2*z-1) ./ (z.* (z-1)) ;
```

%MATLAB支持普通函数向复变函数的自然过渡

```
Path=[2+1i,-1+1i, -1-1i, 2-1i, 2+1i]; %定义积分路径
```

```
sf=integral(F,2+1i,2+1i,'Waypoints', Path)
```

```
sf = 0.0000000000000000 +12.566370614359174i
```

```
err=sf-4*pi*i
```

```
err = 1.665334536937735e-16
```

二重积分的复合中矩体公式

- 如果二重积分的区域为矩形区域，就可以将积分的矩形区域分成若干小的正方形。将总的二重积分分解为每一个矩形区域上的积分和。即：

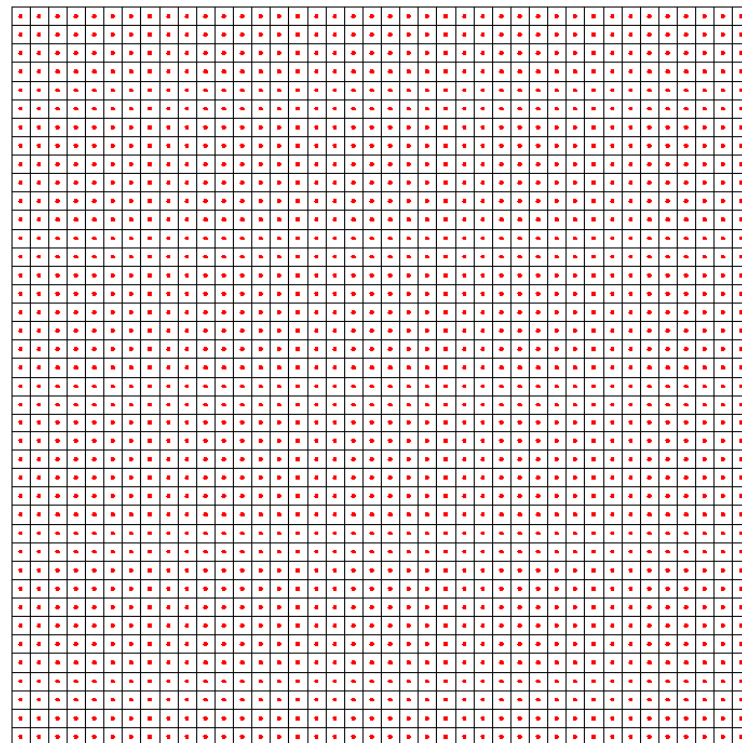
$$\iint_D f(x, y) \, dx dy \approx \sum_{i=1}^m \sum_{j=1}^n \iint_{D_{ij}} f(x, y) \, dx dy$$

- 对每一个小正方形，可以“粗暴”的用中点的函数值乘以正方形的面积，称为复合中矩体公式：

$$\iint_D f(x, y) \, dx dy \approx$$

$$\sum_{i=1}^m \sum_{j=1}^n \iint_{D_{ij}} f(x, y) \, dx dy \approx$$

$$\sum_{i=1}^m \sum_{j=1}^n f(\xi_i, \eta_j) \cdot S_{\square}$$



二重积分的复合中矩体公式

【例】用复合中矩体公式计算 $\iint_D \frac{1}{\sqrt{(1+x^2+y^2)^3}} dx dy$, $\{D: 0 \leq x \leq 1, 0 \leq y \leq 1\}$

%% 中矩体算法：将 $[0, 1] \times [0, 1]$ 横纵坐标分别划分2000份。

f = @(x,y) (1./sqrt((1+x.^2+y.^2).^3));

%匿名函数既可以用于符号运算，也可以用于数值运算

d = 1/2000; x = d/2:d:1-d/2; y = x;

%步长与中矩体采样点的定义，本定义方法旨在不使用for循环。

intmr = sum(sum(f(x',y)))*d*d

%计算积分时，**x**和**y**必须分别为列行向量，确保函数代入后可以形成一个2000 × 2000的矩阵，而后两次sum可以求出整个矩阵的和。最后乘以每一个正方形的面积

intmr =
0.5236

err1 = abs(pi/6 - intmr)

%积分理论值为 $\frac{\pi}{6}$ ，在第三周作业中已有大家尝试

err1 =
8.0188e-09

第八周电子版作业（下周二晚以前提交）

- ❑ 作业请于10月26日（周二）23:59:59或以前提交到课堂派平台。（请同学们加入正确的班级）
- ❑ 作业文件名格式：“05370038；李嘉；第八周作业”
- ❑ 请将所有题目的方法设计（如果有）、代码、运行结果、绘图结果（如果有），放于同一个pdf（推荐使用）文件内（doc或docx也可）。文件名建议与邮件标题完全相同。
- ❑ 作业满分52分，电子版作业迟交者会按照每天5分的梯度损失分数。分数会根据答案的正确与否、算法的高效与否、文档展示的规范性等多个方面综合评定。
- ❑ 请勿抄袭，一经发现，最终总评不超过80。两次抄袭等于挂科。如果进行了部分方法与内容的借鉴，请在作业明示你的参考文献或借鉴人。

第八周电子版作业

- Q1. 改编的习题4第4题（题目有改动！，假设符号计算保留16位小数为真实值，trapz步长自定）

用 integral 求取 $\int_{-5\pi}^{10\pi} e^{-|x|} |\sin x| dx$ 的绝对精度为 10^{-9} 的积分，并尝试用 trapz 及符号计算此积分。以符号计算的结果保留 16 位有效数字作为真实值，分析比较函数 integral 与 trapz 运算的误差（提示：注意 integral 指令相对误差控制对绝对精度的影响。）

- Q2. 根据本次课的课件内容，使用复合中矩形公式，计算定积分 $\int_0^1 \frac{1}{1+x^2} dx$ 的值，并利用积分结果对常数 π 进行近似。

- Q3. 使用 diff 与 gradient 函数，计算 $f(x) = \ln(1+x)$, $0 \leq x \leq 2$ 的近似导数（步长 $x = 1 \times 10^{-5}$ ），然后比较 diff 与 gradient 在 $f'(1)$ 的误差，以分析方法优劣性。（真实值是显而易见的，不需要符号计算☺）。

（未完，下一页还有选做题）

第八周电子版作业

- Q4. 用二维复合中矩体算法计算下列二重积分

$$\iint_D \frac{x+y}{\sqrt{(1+x^2+y^2)^3}} dx dy, \{D: 0 \leq x \leq 1, 0 \leq y \leq 1\}$$

请将积分正方形区域按**2 0 0 0 × 2 0 0 0**的倍数均匀分成四百万个正方形，对每个正方形取中点函数值即可。并用符号运算获得该二重积分的真实值，再转化为双精度后，输出数值计算的误差。（注意：该积分与第二课的积分不同）

- Q5. 利用课件21页下半部分所的二维蒙特卡洛法，设计一套MATLAB代码，完成常数 π 的估计。随机投点个数可以设置为 1×10^6 个。请避免不必要的循环体的使用。

课后反馈（不用交）

- 将今天讲过的例题尝试自己键入并运行一遍
- 阅读数值分析的有关知识，了解并记忆简单中心差分与向前差分近似导数，矩形公式、梯形公式、辛普森共识近似数值积分，以及蒙特卡洛法的简单原理。
- 阅读思考课本对应习题（不作为考点）

感谢同学们认真听课!

欢迎同学们积极提问、交流

mcstj@mail.sysu.edu.cn